

Monetisation and applicability

The safety, observability, and evaluation layers built for this assessment are the same layers a commercial deployment needs. They are not throwaway demo scaffolding — they are the moat.

1. Where the system fits commercially

Four viable models, each with a distinct buyer and pricing shape.

1.1 Embedded analytics feature

Who pays: SaaS vendors — CRMs, ERPs, e-commerce admin panels, ops dashboards. **Where it fits:** add "ask your data" inside an existing product so customers query their own data in natural language without needing SQL. **Pricing:** per-seat add-on or usage-based (metered by query). **Fastest path to monetisation** — the buyer already has customers with data; you're adding a feature to a working product.

1.2 Internal BI copilot

Who pays: enterprises with a data team that gets buried in ad-hoc query requests. **Where it fits:** deflects routine data requests from the queue in front of the data team. Analysts self-serve; engineers focus on hard work. **Pricing:** per-seat, sold to analytics teams or data organisations. **Value prop:** frees senior data engineers from repetitive "can you pull me the numbers for X" tickets.

1.3 Vertical SaaS

Who pays: SMBs by domain — retail chains, clinic networks, logistics operators, restaurant groups, real estate agencies. **Where it fits:** domain-tuned NL analytics with domain-specific few-shots and column docs baked into the retrieval layer. Each vertical is a distinct SKU. **Pricing:** subscription tier by data volume / seat count. **High-margin** because the retrieval layer encodes domain knowledge (e.g. "for a clinic: no-show rate = cancelled_by_patient / total_appointments" or "for retail: fresh inventory = stock where days_in_warehouse < 30") that generic competitors don't have.

1.4 API / usage-based

Who pays: developers building their own analytics UIs. **Where it fits:** NL→SQL as a service, exposed via API. Metered per successful query. Attractive to teams building custom dashboards, embedded reporting features, or internal tooling. **Pricing:** per-query, tiered by SLA (P99 latency, uptime guarantees).

2. The commercial argument — why safety and observability are the moat

Every serious enterprise buyer of an NL-to-SQL system asks the same three questions:

2.1 "Can it destroy or exfiltrate our data?"

Answer this system gives: no, by construction. Not by prompt, not by convention.

- A read-only DB role means writes are refused at the infrastructure layer even if every application-level guardrail fails.
- Chat history lives in MongoDB with no connection string in the agent's environment — a crafted question cannot exfiltrate other conversations.
- Three-layer defence: intent classifier + AST validation + DB role.

Verifiable in 5 seconds via `psql` (see 01_ARCHITECTURE §2.5).

2.2 "Can we audit what it ran and why?"

Answer this system gives: yes, every request is fully traced.

- Per-stage timing, token count, cost for every query
- The SQL that was generated, visible in the UI
- The rows that were returned
- The LLM prompts and outputs at every step
- Langfuse + LangSmith deep-links from every answer
- Every turn persisted to MongoDB as a durable audit trail

Nothing is a black box. The reviewer can walk from a user-facing answer all the way down to the individual LLM token generations that produced it.

2.3 "How do we know its answers are correct?"

Answer this system gives: automated evaluation harness with measurable metrics.

- Four metrics × 31 test cases across positive / negative / neutral / RAGAS
- Runs in 2 minutes; produces a pass-rate report
- **Reproducible by the buyer, not just by the vendor**

See 04_EVALUATION for the numbers and the reproduction commands.

3. The moat is what most demos skip

A working NL→SQL loop is table-stakes. Every AI-first team can build one in a weekend — feed the schema to an LLM, take back SQL, run it, show the rows. That's a demo.

The parts that make it enterprise-buyable are:

- **Safety by construction** (not by prompt) — because "I told the model not to delete data" is not a defensible answer in a compliance review
- **Observability by default** — because a buyer will not sign up for a system they can't audit
- **Evaluation as a CI-runnable artifact** — because "trust us, it works" is not evidence
- **RAG that measurably improves accuracy** — see 04_EVALUATION §3 for a \$86,670 delta on a single realistic question. That is the commercial case for a retrieval layer, in a real number.

These four layers are 80% of the engineering effort and 100% of the commercial defensibility.

4. Extension paths that expand the market

Each extension unlocks a new class of customer.

EXTENSION	UNLOCKS
Multi-database support (Snowflake, BigQuery)	Any company that isn't Postgres-only — i.e. most enterprises
Per-user auth + RBAC	Regulated industries (healthcare, finance) where row-level security is non-negotiable
Vertical fine-tuning of few-shots	Vertical SaaS — each domain becomes a distinct SKU (retail-tuned / clinic-tuned / logistics-tuned)
Auto-charting	Buyers who compete on user-experience against BI tools like Tableau or Metabase
Feedback loop (self-improving RAG)	Long-term stickiness — the tool gets better the more it's used, competitors can't catch up
Clarification loop	Enterprise buyers who need "reliable across edge cases" rather than "great most of the time"

5. Pricing anchors — for reference

Rough market rates for context (Aug 2026):

- **Embedded analytics features** (per-seat add-on) — \$10–50/seat/month for the feature; higher if it's the primary value driver
- **Internal BI copilot** (per-seat, sold to analytics teams) — \$50–200/seat/month
- **Vertical SaaS** (subscription tier) — \$500–5,000/tenant/month depending on data volume and seat count
- **API / usage-based** — \$0.01–0.10/query at wholesale, \$0.05–0.50 at retail (depending on model tier and query complexity)

The unit-economic driver is **LLM cost per query** ($\approx$ \$0.003 for this system today; see 04_EVALUATION §8). With semantic caching (deferred, see 06_EXTENSION_ROADMAP §3.2) this drops another 30–70%, making retail per-query pricing very healthy margin.

6. Why this delivery is a commercial artefact, not just a demo

The three questions in §2 are the ones any enterprise procurement will ask. This delivery answers all three with hard evidence:

- **Safety** — verifiable via 5-second `psql` test
- **Observability** — visible in the UI's trace panel on every answer + deep-link to Langfuse
- **Evaluation** — reproducible via `docker compose exec backend python -m eval.run` in 2 minutes

That evidence is what turns a technical demo into something a buyer can defend to their own procurement, security, and compliance teams.